Interquery Parallelism

1 Overview

In **interquery parallelism**, different queries or transactions execute in parallel with one another. Unlike intraquery parallelism which aims to speed up a single task, interquery parallelism focuses on system-wide efficiency.

Objective: Scalability

The primary use of interquery parallelism is to **scale up** a transaction-processing system to support a larger number of transactions per second (higher throughput).

- **Impact:** Increases throughput significantly.
- **Limitation:** Individual transaction response times are no faster than in a serial machine.

2 Architecture-Specific Implementation

The ease of supporting interquery parallelism varies drastically depending on the hardware architecture:

- **Shared-Memory Systems:** This is the easiest form to support. Since sequential database systems already handle concurrent processing (multi-threading), they can be ported to shared-memory architectures with minimal changes.
- **Shared-Disk/Shared-Nothing Systems:** Much more complex. Requires coordination for tasks like locking and logging via message passing and synchronization across processors.

3 The Cache-Coherency Problem

In distributed or parallel environments with private buffer pools, a major challenge is ensuring all processors see the latest version of data. This is known as the **Cache-Coherency Problem**.

Challenge

When a processor P_1 updates a page in its local buffer, processor P_2 must not be allowed to read a stale version of that page from its own buffer or from the disk.

4 Integrated Cache-Coherency Protocol

To solve this, cache-coherency protocols are often integrated with **Concurrency-Control** mechanisms. A common protocol for shared-disk systems involves:

Protocol Steps

1. **Lock and Read:** Before any access (read/write), a transaction must obtain a **lock** (shared/exclusive). Immediately after acquiring the lock, it reads the **most recent copy** from the shared disk.
2. **Flush and Release:** Before releasing an *exclusive lock*, the transaction must **flush** the modified page back to the shared disk to ensure persistent visibility for the next locker.

4.1 Advanced Performance Optimizations

Modern systems (like Oracle or Oracle Rdb) use even more complex protocols to avoid excessive disk I/O. Instead of flushing to disk every time, a processor may obtain the most recent version of a page directly from another processor's buffer pool via the interconnect.

4.2 Extending to Shared-Nothing

In Shared-Nothing architectures, each page has a **Home Processor** (P_i) responsible for its physical disk storage. Other processors wanting to access that page must send requests to P_i, which then acts as the coordinator using the cache-coherency protocols described above.

Interquery parallelism is the cornerstone of modern high-performance transaction processing systems like Oracle.